{EPITECH}

SUMMER SCHOOL_

< PART 2 BOOTSTRAP - INTRODUCTION TO REACT />

SUMMER SCHOOL

< WEB DEVELOPMENT: THE COMPONENT ERA />

> ”The secret to building large apps is never build large apps. Break your applications into small pieces. Then, assemble those testable, bite-sized pieces into your big application.”
> — **Justin Meyer**

Introduction

In Part 1, you learned how to structure a webpage using HTML and style it using CSS. However, writing everything in a single `index.html` file quickly becomes repetitive and difficult to maintain.

Welcome to React. React is a JavaScript library that allows you to create reusable pieces of code called **Components**.

This bootstrap is a safe playground. You will not build the newspaper here. Instead, you will learn the mechanics of React: how to set up an environment, how to write JSX, and how to pass data between components. Once you master this, the main project will just be a matter of applying these concepts.

Step 1: The React Environment (Vite)

You can no longer just double-click an `index.html` file to see your website. React needs to be translated (compiled) into standard JavaScript that the browser can understand. We use a tool called **Vite** to do this.

Open your terminal, navigate to your workspace, and run these commands to create your sandbox:

```
# 1. Create a new React project named "react-playground"
npm create vite@latest react-playground -- --template react

# 2. Enter the new folder
cd react-playground

# 3. Install the required dependencies
npm install

# 4. Start the local development server
npm run dev
```

Open the URL provided in your terminal (usually `http://localhost:5173`). You should see the default Vite React application running.

Step 2: The Global Stylesheet

We want to focus on React today, not CSS. However, an unstyled page is hard to read. We are providing you with a base stylesheet so your components look great immediately.

1. Open your project in your code editor.
2. Go to the `src/` folder and delete the `App.css` file completely.
3. Open `index.css`, delete everything inside, and paste the following code:

```css
/* src/index.css */
body {
  font-family: 'Arial', sans-serif;
  background-color: #f8fafc;
  color: #0f172a;
  margin: 0;
  padding: 40px;
  display: flex;
  flex-direction: column;
  align-items: center;
}

h1 {
  color: #1e293b;
  margin-bottom: 30px;
}

.custom-btn {
  background-color: #3b82f6;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
  margin: 5px;
  transition: background-color 0.2s;
}

.custom-btn:hover {
  background-color: #2563eb;
}

.badge {
  background-color: white;
  border-left: 4px solid #3b82f6;
  padding: 15px 20px;
  margin: 10px;
  border-radius: 0 8px 8px 0;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.05);
  min-width: 250px;
}

.badge h3 {
  margin: 0 0 5px 0;
  font-size: 1.1rem;
}

.badge p {
  margin: 0;
  color: #64748b;
  font-size: 0.9rem;
}
```

4. Open `App.jsx` and replace its entire content with this minimal structure:

```jsx
// src/App.jsx
function App() {
  return (
    <div>
      <h1>Welcome to the React Sandbox</h1>
    </div>
  );
}

export default App;
```

Save your files. Your browser should now display a clean, centered title on a light background.

Step 3: Your First Component & JSX

In React, a component is simply a JavaScript function that returns user interface code. This syntax, which looks like HTML inside JavaScript, is called **JSX**.

The Rules of JSX:

---

1. **Always return a single parent element.** You cannot return two siblings directly.
2. **Use `className` instead of `class`.** Because `class` is a reserved word in JavaScript, React uses `className` for CSS.
3. **Component names must start with a Capital Letter.**

Let's create a reusable button.
1. Inside the `src/` folder, create a new folder called `components/`.
2. Inside `components/`, create a file called `AlertButton.jsx`.
3. Write the following code:

```jsx
// src/components/AlertButton.jsx

function AlertButton() {
    return (
        <button className="custom-btn">
            Click Me!
        </button>
    );
}

export default AlertButton; // This allows other files to use this component
```

Importing the Component

Now, let's use your new button in your main application. Open `App.jsx` and modify it:

```
import './App.css';
import AlertButton from './components/AlertButton'; // 1. Import it

function App() {
  return (
    <div>
      <h1>Welcome to the React Sandbox</h1>
      <AlertButton /> {/* 2. Use it like a custom HTML tag! */}
      <AlertButton />
      <AlertButton />
    </div>
  );
}

export default App;
```

Tip: Don't forget to import your components before using them. React needs to know where to find the code for your custom elements.

```
import AlertButton from './components/AlertButton';
```

Look at your browser. You just rendered three beautiful blue buttons by reusing a single component!

Step 4: The Power of Props

Rendering identical buttons is not very useful. What if we want the buttons to have different text? This is where **Props** (Properties) come in.

Props allow you to pass custom data into a component, exactly like HTML attributes.

Open your `AlertButton.jsx` and modify it to accept props:

```
// We add "props" as an argument to the function, you can name it anything, but "props" is the convention.
function AlertButton(props) {
    return (
        // We use curly braces { } to inject JavaScript variables into JSX
        <button className="custom-btn">
            {props.text}
        </button>
    );
}

export default AlertButton;
```

Now, go back to your `App.jsx` and pass different data to each button:

```
import './App.css';
import AlertButton from './components/AlertButton';

function App() {
  return (
    <div>
      <h1>Welcome to the React Sandbox</h1>
      <AlertButton text="Save Article" />
      <AlertButton text="Delete" />
      <AlertButton text="Publish Now" />
    </div>
  );
}

export default App;
```

Check your browser. You now have three completely different buttons powered by a single component!

Destructuring Props: A cleaner way to write components is to ”destructure” the props directly in the function arguments.
Instead of `function AlertButton(props)` and using `{props.text}`, you can write:
`function AlertButton({ text })` and just use `{text}`.

Step 5: The Final Assembly

You now have 90% of the knowledge required to complete the newspaper migration project. You know how to:
1. Initialize a React environment.
2. Create isolated components in their own files.
3. Export and Import components.
4. Pass dynamic data using Props.

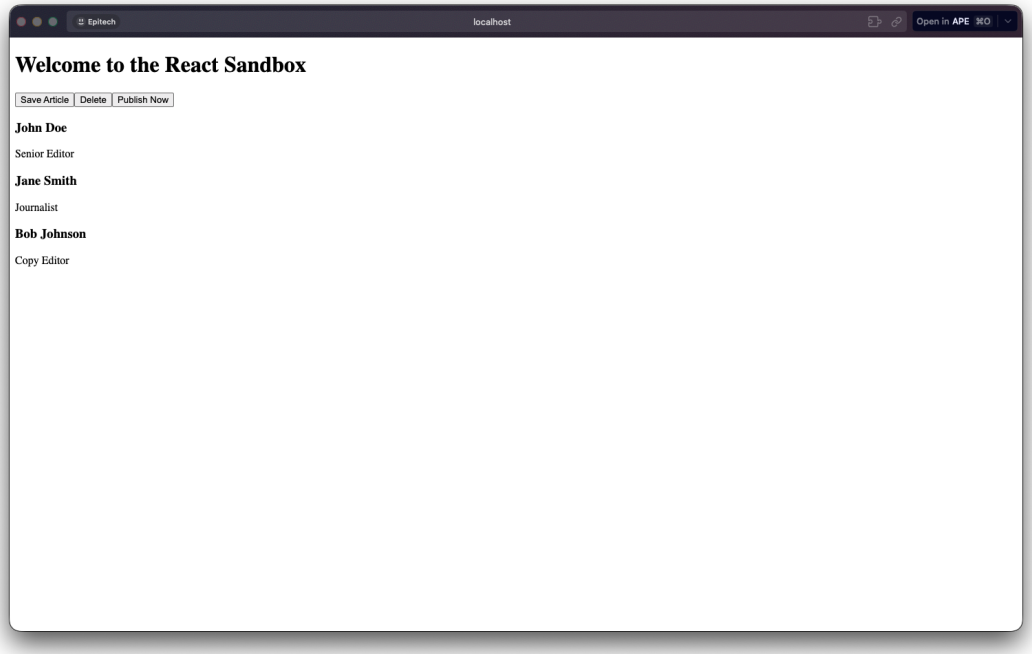
Screenshots of the Final Result

Modern webpages rely on CSS to look good. Here are two screenshots of the same webpage, one without any CSS applied and one with the provided CSS applied.

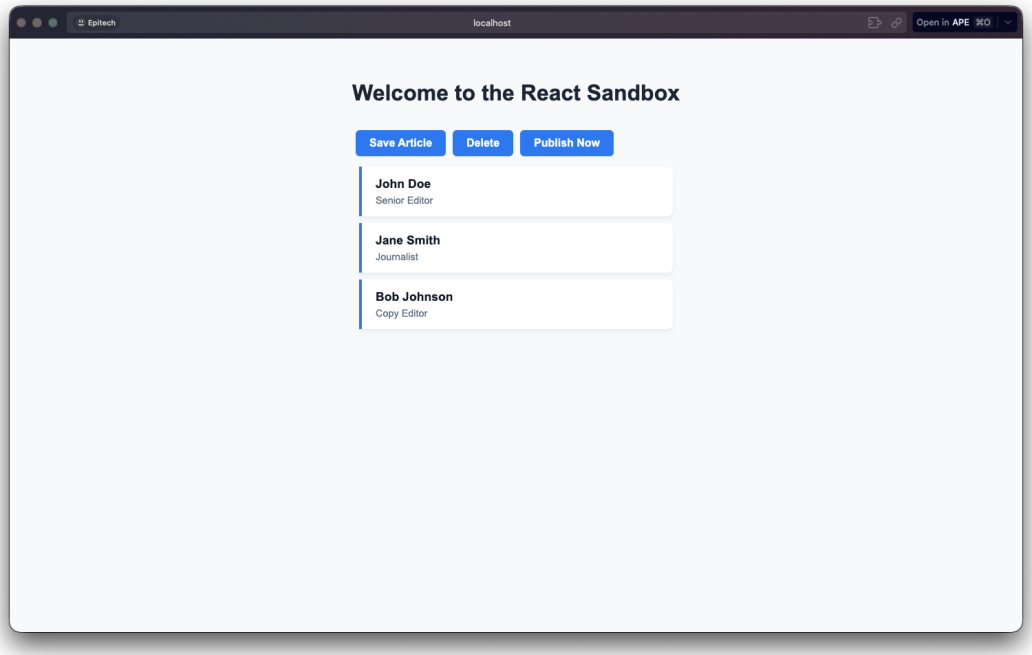
Frameworks like React are often paired with CSS frameworks like Bootstrap or Tailwind. These frameworks provide pre-built styles and components, allowing developers to create visually appealing applications quickly without writing extensive custom CSS.

On the next page, you will see the final result with and without CSS applied. This will help you understand the importance of styling in web development.

This is the webpage **without** any CSS applied:



This is the webpage **with** the provided CSS applied:



v1